

Operations and Verification Guide

What the system does today, how to run and test it by hand, and exactly how far it is from production. Written against the repository on 19 September 2026, including work that was in the working tree but not yet committed, marked as such.

Status in one line

A tested, crash-safe control plane on Bun with one real model route and an authenticated local API, running a synthetic counter as its only job. No repository tools, no findings, no sandbox, no tenant isolation beyond a bearer token. Not production ready, and the repository says so itself.

The system at a glance

LAYER	WHAT IT DOES	WHERE	STATE
Contracts	Versioned Zod schemas for runs, events, controls, findings, model and oracle seams. JSON Schema emitted for Python consumers. Drift manifest pinned by SHA-256.	packages/contracts	Committed
Foundation	Safe errors, canonical digest encoder, fixture contract, findings admission, event reducer.	packages/foundation	Committed
Model gateway	One validated, journaled dispatch per request. Bounded streams, usage accounting, no retry, no fallback. OpenRouter transport pinned to one model and one upstream.	packages/model	Committed
Journal	Authoritative Postgres record: runs, leases, events, actions, model dispatches, artifacts. Owner and epoch checked in every write.	apps/ghidorah/src/storage	Committed
Checkpoint fence	Postgres trigger on Mastra's own snapshot table. Rejects stale owners, wrong runs, finished runs, deletes.	storage/checkpoint-fence.ts	Committed
Runtime	Mastra durable agent, synthetic model, gateway adapter for the real model, bundle integrity check.	apps/ghidorah/src/runtime	Committed
Executor and tools	Admission, intent, dispatch, effect, commit for a tool call. Two counter tools with empty arguments.	execution/, tools/	Committed
Backend and CLI	Run lifecycle, leases, heartbeat, wall deadline, stop. CLI: init, fixture, recover, inspect, fixture-live, recover-live.	backend.ts, cli.ts	Committed
HTTP API	Loopback server with bearer tokens, per-token tenant and engagement authority, permissions, idempotent run creation, SSE events, artifacts, stop.	apps/ghidorah/src/api	Uncommitted
Frontend client	Typical client over the API. Start, snapshot, stop, events. Refuses non-loopback plain HTTP.	apps/frontend/src/client.ts	Uncommitted

Ghidrah operations and verification guide, branch monorepo/bun-workspaces, 19 September 2026, Not production approval

Mastra patch v2	Adds a third upstream fix in the durable-agent bundle; four bundles now hash-pinned.	scripts/mastra-recovery-patch.mjs	Uncommitted
-----------------	--	-----------------------------------	-------------

Size: 70 TypeScript files, about 10,200 lines including tests. Runtime is Bun 1.4.2 only. Postgres 17 via Docker Compose for development.

PART 1

How it works in production terms

Every guarantee below is stated with the mechanism that provides it and the test that proves it. A guarantee without a named test is not on this list.

GUARANTEE	MECHANISM	PROOF
Nothing unregistered runs	Admission rejects unknown targets, models, capabilities, fields and scopes before a row exists	Unit tests and evals GID-001 to GID-043
The client never sees an uncommitted result	Events are appended inside the same transaction as the state change and read back from Postgres in sequence	Integration: sequence contiguity on every run
One owner per run	Lease with owner UUID and epoch, heartbeat, expiry; every write checks both	Integration: stale lease rejected for reserve, record, complete
Stale workers cannot corrupt Mastra state	Postgres trigger on the snapshot table checks run, owner, epoch, expiry, terminal state	Integration: checkpoint fence races and denied deletes
A crash never repeats an uncertain effect	Action ladder prepared, dispatched, effect, completed; recovery blocks on dispatched without result	Integration: real SIGKILL at five fault points, two block
A model call is billed once and replayed for free	Dispatch journal reserves before the call, commits after, returns the stored response on the same request ID	Integration and live acceptance: replay made zero HTTP calls
A cut provider stream is never a final answer	Client requires finish, usage and the DONE sentinel; gateway records the failure as unresolved	Integration: disconnect leaves durable uncertainty; unit: no-DONE is incomplete
A refusal is a recorded outcome, not a fallback trigger	Refusal completes the dispatch, charges usage, executes nothing, never switches provider	Integration: refusal case
A run cannot be recovered on a different model route	The model profile digest is the run's runtime version	Integration: unconfigured journal rejects a live run
An unpatched Mastra cannot execute	Bundle hashes verified at install and before every run	Recovery suite: installation, rejection, behaviour
Evidence bytes cannot change silently	Artifacts stored with SHA-256, rechecked on read	Unit and integration on artifact reads
Budget is enforced, not estimated	Tokens, steps and wall time charged in the journal; wall deadline armed from persisted remaining time	Integration: wall deadline; eval GID-100 token cap

One run, end to end

1 Admit.

Validate the contract version, caps, target, model and exact scope allowlist.

2 Check integrity.

Hash the installed Mastra bundles; confirm the fence trigger exists.

3 Own.

Write the run row, take a lease, start the heartbeat, arm the wall-clock deadline.

4 Loop.

Mastra asks the model. Synthetic model: recorded and replayed. Real model: gateway reserves budget, calls OpenRouter once, commits usage and the response.

5 Execute.

A proposed tool passes admission, then prepared, dispatched, effect, completed, each a fenced transaction that also appends an event and charges budget.

6 Report.

Events stream in sequence from the journal. A terminal record ends the run. Finding counts are zero by schema.

PART 2

Operating it

Prerequisites

NEED	VALUE	NOTE
Bun	1.4.2 or newer	The only runtime. TypeScript runs directly; <code>tsc</code> is type-check only.
PostgreSQL	17, dedicated database	<code>docker compose up -d</code> binds 127.0.0.1:55432 with a public development password. Never reuse it.
<code>GIDORAH_DATABASE_PROFILE</code>	<code>local</code>	Only accepted value. Remote profiles are rejected.
<code>GIDORAH_DATABASE_URL</code>	loopback Postgres URL	No query or fragment. <code>DATABASE_URL</code> is never read.
<code>OPENROUTER_API_KEY</code>	bearer key	Only for <code>fixture-live</code> and the acceptance script. Read from the process environment only.
<code>GIDORAH_API_TOKENS_JSON</code> , <code>GIDORAH_API_PORT</code>	token list, port	For the uncommitted API server. See Part 4.

Bun loads .env automatically

Bun reads a `.env` in the working directory into `process.env` for every script. No code in this repo opens that file, and the database config still accepts only the explicit variables, but anything you put in `.env` is visible to every command you run. Keep the OpenRouter key there only on machines where every script may use it. `.env` is ignored by Git and has never been committed.

Commands

COMMAND	WHAT IT DOES	NEEDS DB	SPENDS MONEY
<code>bun install --frozen-lockfile</code>	Installs workspaces and applies the Mastra patch through postinstall	no	no
<code>bun run patch:check</code>	Verifies Mastra version and bundle hashes without writing	no	no
<code>bun run db:check / db:init</code>	Read-only connectivity check / create the two marked schemas and the fence	yes	no
<code>bun run cli fixture</code>	Counter fixture on the synthetic model	yes	no
<code>bun run cli fixture-live</code>	Counter fixture on GLM 4.7 via OpenRouter, upstream DeepInfra, 60,000-token cap	yes	yes, about \$0.001
<code>bun run cli recover <id> / recover-live <id></code>	Resume a run after its owner died. Live runs need the live profile.	yes	only if model calls remain
<code>bun run cli inspect <id></code>	Snapshot plus the action table of a run	yes	no
<code>bun run test / test:recovery / test:comparison</code>	Offline suites	no	no
<code>bun run test:integration</code>	Real process kills, fence races, dispatch journal, gateway loop, API	yes	no
<code>bun run eval</code>	100 catalogued cases with source fingerprints	yes	no
<code>bun run repro:native</code>	Kill a native Mastra model call, recover from Postgres	yes	no
<code>bun run acceptance:openrouter</code>	Capped live gateway acceptance, six assertions	yes	yes, under \$0.001
<code>bun run verify</code>	Format, typecheck, contracts, all suites, evals, native repro	yes	no

PART 3

Manual test script

Run in order from the repository root. Each step names what you should see. If a step does not match, stop; later steps assume the earlier ones held.

1 Install and verify the patched dependency.

```
bun install --frozen-lockfile
bun run patch:check
```

Expect: JSON with `"verified":true` and four bundle hashes (two on the committed branch). Any other output means the Mastra bytes are not the accepted ones; nothing will run.

2 Start the database and initialise the schemas.

Ghidorah operations and verification guide · branch monorepo/bun-workspaces · 19 September 2026 Not production approval

```
docker compose up -d
export GIDORAH_DATABASE_PROFILE=local
export GIDORAH_DATABASE_URL='postgresql://gidorah:gidorah_dev_only@127.0.0.1:55432/gidorah'
bun run db:check
bun run db:init
```

Expect: "connected":true, then "Initialized only the gidorah_mastra and gidorah_mastra_runtime fixture schemas". Re-running init is safe; it refuses to adopt unmarked schemas.

3 Run the counter on the synthetic model.

```
bun run cli fixture
```

Expect: Fourteen JSON events on stdout, sequence 1 to 14: run.started, step, tool.call fixture_increment, tool.result ok, budget, tool.call fixture_read, tool.result ok, budget, step, run.finished with "outcome":"completed" and all finding counts 0. Final budget shows 6 tokens and 5 steps. Exit code 0.

4 Inspect it and confirm recovery of a finished run is a no-op.

```
bun run cli inspect <runId>
bun run cli recover <runId>
```

Expect: Inspect shows a snapshot with terminal.outcome completed and two actions both in state completed. Recover prints exactly one run.snapshot event and exits 0.

5 Kill a worker at a safe point and recover.

```
bun apps/ghidorah/test/helpers/crash-worker.ts after-intent
bun run cli recover <runId from the first line>
```

Expect: The worker prints a runId, a few events, then {"faultPoint":"after-intent"} and dies by SIGKILL with no further output. Recovery completes the run; inspect shows the counter at 1, not 2.

6 Kill a worker at an uncertain point and confirm the block.

```
bun apps/ghidorah/test/helpers/crash-worker.ts after-dispatch
bun run cli recover <runId>
```

Expect: Recovery prints a fatal error event with code uncertain_execution and exits 1. The run has no terminal state. Inspect shows one action stuck in state dispatched. This is correct behaviour: a human must reconcile it.

7 Run the counter on the real model.

```
export OPENROUTER_API_KEY=... # or rely on .env under Bun
bun run cli fixture-live
```

Expect: stderr shows "LIVE MODEL: z-ai/glm-4.7 via OpenRouter, upstream DeepInfra". stdout shows tool.call fixture_increment, tool.result ok, tool.call fixture_read, tool.result ok, run.finished completed. Token spend in the hundreds, typically 700 to 1,000, three model steps. Cost about a tenth of a cent.

8 Confirm route binding.

```
bun run cli recover <live runId>
bun run cli recover-live <live runId>
```

Ghidorah operations and verification guide · branch monorepo/bun-workspaces · 19 Not production
Expect: Main 2026 recover fails with runtime_version_mismatch: a process without the live profile cannot read a live run. recover-live prints one snapshot and exits 0.

9 Run the automated proof.

```
bun run verify
```

Expect: All green. On 19 September: format, typecheck, contracts, 148 unit, 8 recovery, 5 comparison, 29 integration plus 5 API, 100 evals, native repro. Total under ten minutes.

PART 4

The HTTP API, uncommitted at time of writing

A loopback-only server that turns the in-process backend into something a frontend can call. It exists in the working tree with five passing integration tests and is not yet committed. Treat the shapes below as current, not final.

Start it

```
TOKEN=$(openssl rand -base64 48 | tr -d '=/+' | cut -c1-48)
HASH=$(printf %s "$TOKEN" | shasum -a 256 | cut -d' ' -f1)
export GIDORAH_API_TOKENS_JSON='[{"tokenSha256":"'$HASH'", "principal":{"tenantId":"tenant-a", "actorId":"aditya", "engagementId":"eng-1", "policyRevision":"p1", "permissions":["run:create", "run:read", "run:control"], "expiresAt":$(( $(date +%s) * 1000 + 3600000 ))}]'
bun run api:fixture      # listens on 127.0.0.1:4317
```

Endpoints

METHOD AND PATH	PERMISSION	BEHAVIOUR
POST /v1/runs	run:create	Body { target, config }. Header Idempotency-Key must be a UUID; the same key with the same body returns the same run, a different body is rejected. Responds 202 with { runId, created }.
GET /v1/runs/:id	run:read	Current snapshot. Only for runs created under the caller's tenant and engagement authority.
GET /v1/runs/:id/events	run:read	Server-sent events: event: record, id: seq, data: {event}. Snapshot first, then live. At most 16 concurrent observers.
GET /v1/runs/:id/artifacts?ref=sha256:...	run:read	Digest-checked evidence bytes with a redaction flag.
POST /v1/runs/:id/control	run:control	Body { contractVersion, type: "stop" }. Responds 202.

Any request carrying an Origin header is refused with 403; browsers are deliberately not enabled yet. Tokens are compared by SHA-256 in constant time and expire. The server binds 127.0.0.1 only and has no TLS.

Try it

```
curl -s -X POST 127.0.0.1:4317/v1/runs -H "authorization: Bearer $TOKEN" \
  -H "idempotency-key: $(uuidgen | tr A-Z a-z)" -H "content-type: application/json" \
  -d '{"target":"fixture://counter","config":{"contractVersion":"1.0.0","model":"gidorah-fixture-v1","targetKind":"web","capabilities":["agentic_pentesting"],"capSteps":500,"capTokens":1000,"capWallSec":7200,"approvalProfile":"closed-world","authorization":{"asserted":true,"scopeAllowlist":["fixture://counter"]}}}'
curl -N 127.0.0.1:4317/v1/runs/<runId>/events -H "authorization: Bearer $TOKEN"
```

Expect: 202 with a runId, then an SSE stream ending in a `run.finished` record with outcome completed. A token from another tenant gets 404 on the same run; a read-only token gets 403 on POST.

PART 5

Production readiness

The repository's own plan lists fourteen release gates and marks release blocked. This is where each one stands after this week's work.

GATE	STATUS	WHAT EXISTS	WHAT IS MISSING
P1 Foundation recovery	Partial	Real kills at five fault points, fence races, gateway disconnect case, native repro	Two-process takeover race, multi-hour endurance, storage outage and restore
P2 Core contract	Partial	Schemas, manifest, frontend guide, client	Consumer acceptance by frontend, Gyms, Model; durable finding and receipt tables
P3 Model adapter	Largely done	Gateway, Postgres dispatch journal, OpenRouter route, Mastra wiring, live run	OpenAI adapter, tokenizer-based reservation, USD ledger
P4 Gyms verification seams	Not started	Schemas only	Authenticated verify and ranges calls, attempt journal
P5 Isolated executor	Not started	Counter tools only	Sandbox, egress control, escape tests
P6 Cleanup lifecycle	Not started	Counter only	Durable allocation and mutation intents, reconciliation
P7 Evidence and data security	Not started	Digest-checked numeric artifacts	Redaction, encryption, retention
P8 Engagement access	Started	Bearer tokens with tenant and engagement authority on runs, uncommitted	Real identity provider, audit, least-privilege DB roles, TLS
P9 Approvals and reviews	Not started	Schema-valid, rejected at runtime	Persisted approvals, recovery semantics
P10 Headless and frontend contract	Started	Loopback API with SSE, typed client, exit-code policy in CLI	Reconnect protocol, final result document, report export
P11 M0 acceptance	Not started		End-to-end protected task with independent verdict
P12 Authorization pilot	Not started		Gyms predicates, permission policy
P13 Customer release	Blocked		Everything above, plus deployment, backups, monitoring, incident drill
P14 Owned model	Separate track		Not required for the pilot

Risks worth naming

Production readiness and verification guide · branch monorepo/bun-workspaces · 19 Not production approval
September 2026

RISK	WHY IT MATTERS	MITIGATION IN PLACE
------	----------------	---------------------

Mastra is patched, not upstream	Every Mastra upgrade re-opens the recovery bugs	Hash pins at install and run; upgrade procedure documented
Mastra's Bun support is not vendor-tested	A Bun-only bug would be ours to find	Full suite passes on Bun; keep it in CI
OpenRouter load-balances upstreams	One upstream ignored the output cap	Upstream pinning mandatory; unpinned routes refused
Input reservation is a byte heuristic	Over-reserves; blocks small budgets	Versioned in the profile; swap for a tokenizer later
Postgres is the only store	Install friction for a CLI product	Deliberate: the fence and leases need it

Next three steps toward a shippable product

- 1 Read-only repository tools.**
 Behind the existing executor. Closed world, no containment needed. Unlocks code review.
- 2 A finding producer.**
 Persisted candidates and receipts. Lights up the frontend's finding events.
- 3 Commit and harden the API.**
 TLS termination, real identity, audit log, then the reconnect protocol.